

Assignment #3

DS-3003

DATA WAREHOUSING & BUSINESS INTELLIGENCE



DEADLINE: 3rd December 2024

Total Marks: 100, Weightage: 5%

Course coordinator: Dr. Muhammad Asif Naeem

Important Instructions:

1. Try to complete the assignment before the deadline.
2. Try to attempt all questions! Data warehouse is a highly useful and fast-growing field. This assignment will help you understand the basics very well.
3. Plagiarism is strictly prohibited! Try to solve the assignment on your own. Marks will be given based on your thought process so make sure you have a solid reason for all your attempts.
4. The assignment is of 100 marks in total. Individual marks for each question and part are mentioned below alongside the questions.
5. The assignment should be submitted in a document format. For diagrams, it can be handwritten. No assignment will be accepted after the due date.

Question 1: Optimizing Multi-Table Join in a Student Database. (20 marks)

Consider a student database for a university system consisting of the following five tables:

Students: StudentID (Primary Key), FirstName, LastName, DateOfBirth, Gender,

Courses: CourseID (Primary Key), CourseName, DepartmentID (Foreign Key), Credits

Departments: DepartmentID (Primary Key), DepartmentName

Enrollments: EnrollmentID (Primary Key), StudentID (Foreign Key), CourseID (Foreign Key), EnrollmentDate

Grades: GradeID (Primary Key), EnrollmentID (Foreign Key), GradeValue

Compose a query that retrieves the following information for each student:

- Student's full name
- Department they are enrolled in
- Courses they are enrolled in
- Credits for each course
- Grades received in each course

The query should involve joining all five tables to retrieve the required information.

Requirements:

- Optimize the query for performance, considering the complexity of the join.
- Propose appropriate indexing strategies to enhance join efficiency.
- Discuss the potential use of materialized views to improve query performance.
- Outline any partitioning recommendations to optimize the join operation.
- Provide a comprehensive explanation of your optimization decisions.

Question 2: (20 marks)

In the All-Electronics data warehouse, suppose the dimension item at the top level has four values (representing item types): “home entertainment,” “computer,” “phone,” and “security.” Each value (e.g., “computer”) is represented by a bit vector in the item bitmap index table. Suppose that the cube is stored as a relation table with 100,000 rows. Because the domain of the item consists of four values, the bitmap index table requires four-bit vectors (or lists), each with 100,000 bits. Figure shows a base (data) table containing the dimension’s item and city. **Create a bitmap table for each of the dimensions.**

RID	Item	City
R1	H	T
R2	C	V
R3	P	T
R4	S	V
R5	H	T
R6	C	V
R7	P	T
R8	S	V

- H: "Home Entertainment"
- C: "Computer"
- P: "Phone"
- S: "Security"
- V: "Vancouver"
- T: "Toronto"

Question 3: (20 marks)

Given this array {10, 9, 4, 3, 22, 28, 5, 2, 17, 11, 18, 45, 67, 45, 58, 12, 38}

1. Create a 5-way B tree. (10 marks)
2. Draw R-tree with MBR (Minimum Bounding Range). (10 marks)

Question 4: (40 marks)

Scenario:

You are tasked with optimizing a large dataset in a Data Warehousing environment through partitioning. The dataset represents sales transactions with various attributes. Your goal is to implement both vertical and horizontal partitioning to improve data retrieval performance.

Dataset Details:

Consider a table named "SalesTransactions" with the following columns:

1. **TransactionID**: A unique identifier for each sales transaction.
2. **ProductID**: A unique identifier for the products sold in each transaction.
3. **CategoryID**: A classification identifier for product categories.
4. **CustomerID**: A unique identifier for the customers involved in the transactions.
5. **TransactionDate**: The date and time when each sales transaction occurred.
6. **Quantity**: The quantity of products sold in each transaction.
7. **UnitPrice**: The price of each unit of the product.
8. **Discount**: Any discounts applied to the transaction.
9. **TotalAmount**: The total amount paid for the transaction.
10. **Region**: The geographical region associated with the sales transaction.

Task 1: Vertical Partitioning (20 marks)

- Identify columns that could be candidates for vertical partitioning based on their frequency of use.
- Create a new table, "FrequentlyUsedData," to include selected columns. Write sql query for the table creation.
- Create another table, "SeldomUsedData," to store the remaining columns from the original table. Write sql query for the table creation.
- Write sql queries that would transfer data into these newly created tables.

Task 2: Horizontal Partitioning (20 marks)

- How will you do horizontal partitioning on the "SalesTransactions" table based on the "TransactionDate" column. (Consider the dataset is of 2 years, from January, 2020 to January 2022. The dataset should be partitioned into two horizontal partitions based on a specified date range.
- Write sql queries that would partition and transfer the data.